Go2 SDK Development Guide

Search article titles

About Go2
Remote Controller
SDK Concepts 3
Application Development 7
Software Interface Services 19
Motion Services Interface V2.0
DDS Services Interface
Avoidance Services Interface
VuiClient Interface
RobotStateClient Interface
Motion Switcher Service Interface
Sports Services Interface
AI motion Services interface
Basic Services Interface
LiDAR Services Interface
Multimedia Services Interface
UWB Services Interface
Fault Services Interface
SLAM and Navigation Services Interfa
Aruco Recharge Service Interface
Odometer Services Interface
D1 Mechanical Arm Services Interface
ROS2 Services Interface
Python Services Interface
Case reference 8
Debugging Specification
FAQ

Basic Services Interface

Last Updated On : 2025-11-21 11:19:49

The underlying communication mainly involves obtaining data from the motor, battery, remote control, and IMU, publishing **rt/lowstate**, subscribing to control commands **rt/lowcmd**, and controlling the motor and battery.

Interface Description

Lower level control instructions

Users can send control commands for motors, batteries, automatic charging, and motor power switches by subscribing to the DDS topic "**rt/lowcmd**". The data format is **LowCmd_.idl**.

- **Motor control command**: There are a total of 12 motors on the body, and the control sequence can be carried out as follows. For details, please refer to **MotorCmd_.idl**.
- **Battery Control Command**: Turn off the battery on the body of the machine. For details, refer to **BmsCmd_.idl**.
- **Automatic Charging Command**: A command to control automatic charging, where **&0xFE**->automatic charging is turned on and **| 0x01**->automatic charging is turned off.
- **Motor Power Control Command**: A command to control the power supply of the motor. **&0xFD** ->The power switches of the 12 motors are turned on, and **|0x02** ->the power switches of the 12 motors are turned off.

```
// generated from rosidl_generator_dds_idl/resource/idl.idl.em
    // with input from unitree_go:msg/LowCmd.idl
    // generated code does not contain a copyright notice
    #include "BmsCmd_.idl"
    #include "MotorCmd_.idl"

    #ifndef __unitree_go__msg__low_cmd__idl__
    #define __unitree_go__msg__low_cmd__idl__


    module unitree_go {

    module msg {

    module dds_ {


    struct LowCmd_ {
    octet head[2];              // Frame header, used for data verification (0xFE, 0xEF).

    octet level_flag;           //Reserved, currently not used.
    octet frame_reserve;        //Reserved, currently not used.
    unsigned long sn[2];        //Reserved, currently not used.
    unsigned long version[2];   //Reserved, currently not used.
    unsigned short bandwidth;   //Reserved, currently not used.

    // FR_0 -> 0 , FR_1 -> 1  , FR_2 -> 2   The motor control sequence is currently only 12 motors, which will l
    // FL_0 -> 3 , FL_1 -> 4  , FL_2 -> 5
    // RR_0 -> 6 , RR_1 -> 7  , RR_2 -> 8
    // RL_0 -> 9 , RL_1 -> 10 , RL_2 -> 11
    unitree_go::msg::dds_::MotorCmd_ motor_cmd[20];   //Motor control command data
    unitree_go::msg::dds_::BmsCmd_ bms_cmd;           // Battery control command data

    octet wireless_remote[40];  //Reserved, currently not used.
    octet led[12];              //It has been changed to internal control and is currently not used
    octet fan[2];               //It has been changed to internal control and is currently not used

    //&0xFE automatic charging on, | 0x01 automatic charging off
    //&0xFD 12 motor power switches on, | 0x02 12 motor power switches off    octet gpio;

    unsigned long reserve;  // reserved
    unsigned long crc;      // For data CRC verification, it is used for 32crc verification.

    };


    };  // module dds_

    };  // module msg

    };  // module unitree_go


    #endif  // __unitree_go__msg__low_cmd__idl__
```

MotorCmd_.idl

Real time information of motor control commands for motion control.

```
#ifndef __unitree_go__msg__motor_cmd__idl__
#define __unitree_go__msg__motor_cmd__idl__


module unitree_go {

module msg {

module dds_ {


struct MotorCmd_ {
```

This page contains content

Interface Description
Lower level control instructions
Low level data reception

Helpful
Feedback

```
octet mode;  // Motor control mode (Foc mode (working mode) ->0x01, stop mode (standby mode) ->0x00.)
float q;     //Joint target position
float dq;    //Joint target speed
float tau;   //Joint target torque
float kp;    //Joint stiffness coefficient
float kd;    //Joint damping coefficient
unsigned long reserve[3];   //reserved


};


};  // module dds_

};  // module msg

};  // module unitree_go


#endif  // __unitree_go__msg__motor_cmd__idl__
```

BmsCmd_.idl

Command to turn off the body battery.

```
// generated from rosidl_generator_dds_idl/resource/idl.idl.em
// with input from unitree_go:msg/BmsCmd.idl
// generated code does not contain a copyright notice

#ifndef __unitree_go__msg__bms_cmd__idl__
#define __unitree_go__msg__bms_cmd__idl__


module unitree_go {

module msg {

module dds_ {


struct BmsCmd_ {
octet off;          // Turn off battery: (Command: 0xA5)
octet reserve[3];  //reserve


};


};  // module dds_

};  // module msg

};  // module unitree_go


#endif  // __unitree_go__msg__bms_cmd__idl__
```

Low level data reception

Users can obtain motor, battery, IMU, and remote control data by publishing the DDS topic "**rt/lowstate**" in the format of **LowState_.idl**.

- **Motor status information**: A total of 12 motors can be obtained in the following order. Please refer to **MotorState_.idl** for details.
- **IMU status information**: includes three axis acceleration and angular velocity information, quaternion information, Euler angle information, and temperature information. Please refer to **IMUState_.idl** for details.
- **Battery Status Information**: Contains battery version, status information, battery level information, charging and discharging, cycle times, temperature, and single battery voltage. For details, please refer to **BmsState_.idl**.
- **foot_force**[4]: represents the foot force information of each leg. Order (0-FR，1-FL，2-RR, 3-RL).
- **bit_ Flag**: The status information of each component, used for real-time detection of the status of each component. (1. Detect whether the motor, small MCU, remote control, battery, and operation command have timed out. 2. Detect whether there is an error flag in the onboard current, and trigger the hardware current protection to generate an error flag. 3. Detect whether it is in the automatic charging state.).
- **fan_frequency**[4]: Fan speed and locked rotor detection (0-Left rear speed, 1-Right rear speed, 2-Front speed, in revolutions per minute) (locked rotor detection: 3-&0x01: Left rear locked rotor,&0x02: Right rear locked rotor,&0x04: Front locked rotor).
- **Protection Mechanism**: **adc_reel**- >Winder current, **temperature_ntc1**->motherboard center temperature value, **temperature_ntc2**->Automatic charging temperature, **power_v**->Battery voltage, **power_a**->Motor current.

```
// generated from rosidl_generator_dds_idl/resource/idl.idl.em
// with input from unitree_go:msg/LowState.idl
// generated code does not contain a copyright notice
#include "BmsState_.idl"
#include "IMUState_.idl"
#include "MotorState_.idl"

#ifndef __unitree_go__msg__low_state__idl__
#define __unitree_go__msg__low_state__idl__


module unitree_go {

module msg {

module dds_ {


struct LowState_ {
octet head[2];              // Frame header, used for data verification (0xFE, 0xEF).

octet level_flag;           //It is still used, but not currently used.
octet frame_reserve;        //It is still used, but not currently used.
unsigned long sn[2];        // It has been changed to file storage format and is currently not used.
```

```c
unsigned long version[2];    //It is still used, but not currently used.
unsigned short bandwidth;    //It is still used, but not currently used.。

unitree_go::msg::dds_::IMUState_ imu_state;           // IMU data information.

//FR_ 0->0, FR_ 1->1, FR_ 2->2 motor sequence, currently only 12 motors are used, later reserved.
//FL_ 0->3, FL_ 1->4, FL_ 2->5
//RR_ 0->6, RR_ 1->7, RR_ 2->8
//RL_ 0->9, RL_ 1->10, RL_ 2->11

unitree_go::msg::dds_::MotorState_ motor_state[20];   // Motor total data.
unitree_go::msg::dds_::BmsState_ bms_state;           // Battery total data.

short foot_force[4];        //Foot force (range 0-4095), can be displayed according to actual values. (Arrays: (
short foot_force_est[4];    //It is still used, but not currently used.

unsigned long tick;         //1ms计时用，按照1ms递增。
octet wireless_remote[40];  //遥控器原始数据。

//&0x80- Motor timeout flag 1- timeout 0- normal
//&0x40- Small Mcu timeout flag 1- timeout 0- normal
//&0x20- Remote control timeout flag 1- timeout 0- normal
//&0x10- Battery timeout flag 1- timeout 0- normal
//&0x04- Automatic charging automatic charging status flag 1- Not charging 0- Charging
//&0x02- On board current error flag Error flag 1- On board current abnormal 0- Normal
//&0x01- Operation control command timeout timeout flag 1- timeout 0- normal

octet bit_flag;             // Display of status of each component

float adc_reel;             // Winder current (range: 0-3A).
octet temperature_ntc1;         // The central temperature value of the motherboard (range: -20-100 ℃).
octet temperature_ntc2;         // Automatic charging temperature (range: -20-100 ℃).
float power_v;                                  // This voltage value is the motherboard voltage ->battery volta
float power_a;                                  // This current value is the motherboard current value ->motor (

unsigned short fan_frequency[4];    // Fan speed (currently displaying 0-10000 based on actual values). (0- Lefi

unsigned long reserve;   //reserved。
unsigned long crc;       // Used for data CRC verification.

};


};  // module dds_

};  // module msg

};  // module unitree_go


#endif  // __unitree_go__msg__low_state__idl__
```

IMUState_.idl

1. **quaternion**[4]: Provides real-time quaternion information for the body posture. (0-w, 1-x, 2-y, 3-z).

2. **rpy**[3]: Provides real-time Euler angle information for the body posture of the aircraft. (0-roll, 1-switch, 2-yaw).

3. **gyroscope**[3]: Provides real-time three-axis angular velocity information of the fuselage posture. (0-x, 1-y, 2-z).

4. **accelerometer**[3]: Provides real-time three-axis acceleration information of the fuselage posture information. (0-x, 1-y, 2-z).

```c
// generated from rosidl_generator_dds_idl/resource/idl.idl.em
// with input from unitree_go:msg/IMUState.idl
// generated code does not contain a copyright notice

#ifndef __unitree_go__msg__imu_state__idl__
#define __unitree_go__msg__imu_state__idl__


module unitree_go {

module msg {

module dds_ {


struct IMUState_ {
float quaternion[4];    // Quaternion data

float gyroscope[3];     // Angular velocity information (0->x, 0->y, 0->z)

float accelerometer[3]; // Acceleration information (0->x, 0->y, 0->z)

float rpy[3];           //Euler angle information: defaults to the radian value (which can be changed to the ang

octet temperature;      //IMU temperature information in degrees Celsius.

};


};  // module dds_

};  // module msg

};  // module unitree_go


#endif  // __unitree_go__msg__imu_state__idl__
```

MotorState_.idl

Real time information fed back by the motor for motion control.

```c
// generated from rosidl_generator_dds_idl/resource/idl.idl.em
// with input from unitree_go:msg/MotorState.idl
// generated code does not contain a copyright notice

#ifndef __unitree_go__msg__motor_state__idl__
#define __unitree_go__msg__motor_state__idl__
```

```
module unitree_go {

module msg {

module dds_ {


struct MotorState_ {
octet mode;     //Motor control mode (Foc mode (working mode) ->0x01, stop mode (standby mode) ->0x00.)
float q;        // Joint feedback position information: Default is in radians (can be changed to degrees as nee
float dq;       //Joint feedback speed
float ddq;      //Joint feedback acceleration
float tau_est;  //Joint feedback torque

float q_raw;    //It is still used, but not currently used.
float dq_raw;   //It is still used, but not currently used.
float ddq_raw;  //It is still used, but not currently used.
octet temperature;          //Motor temperature information: Type: int8_t. It can be displayed according to actu
unsigned long lost;         //Motor packet loss information: can be displayed according to actual values (range.
unsigned long reserve[2];   //Current motor communication frequency+motor error flag bits: (Array: 0- Motor erro


};


};  // module dds_

};  // module msg

};  // module unitree_go


#endif  // __unitree_go__msg__motor_state__idl
```

BmsState_.idl

- **version_high** and **version_low**: Information on the version of the constituent battery.
- **status**: Display states such as battery not turned on, wake-up event, battery pre charging, battery charging normally, battery disc harging normally, battery self discharging, battery presence warning, waiting for button reset warning, and resetting.
- **soc**: Battery level information (1% -100%).
- **current**: Charging and discharging information:
  - Positive: represents charging.
  - Negative: represents discharge.
- **cycle**: The number of charging cycles.
- **bq_ntc**[2]: The temperature of the two NTCs inside the battery
  - 0-BAT1
  - 1-BAT2
- **mcu_ntc**[2]: Battery NTC array
  - 0-RES
  - 1-MOS
- **cell_vol**[15]: Single section voltage.

```
// generated from rosidl_generator_dds_idl/resource/idl.idl.em
// with input from unitree_go:msg/BmsState.idl
// generated code does not contain a copyright notice

#ifndef __unitree_go__msg__bms_state__idl__
#define __unitree_go__msg__bms_state__idl__


module unitree_go {

module msg {

module dds_ {


struct BmsState_ {
octet version_high;    //Battery version
octet version_low;     //Battery version

//0: SAFE, (battery not turned on)
//1: WAKE_ UP, (wake up event)
//6: PRECHG, (during battery pre charging)
//7: CHG, (during normal battery charging)
//8: DCHG, (during normal battery discharge)
//9: SELF_ DCHG (during battery self discharge)
//11: ALARM, (battery presence warning)
//12: RESET_ ALARM, (waiting for button reset warning)
//13: AUTO_ RECOVERYOctet status // Battery status information.
octet soc;             // Battery level information: (Type: uint8_t) (Range 1% -100%)
long current;          // Charging and discharging information: (positive: represents charging, negative repres
unsigned short cycle;  // Number of charging cycles
octet bq_ntc[2];       // The temperature of the two NTCs inside the battery (int8_t) (range: -100-150). 0- BAT1

octet mcu_ntc[2];      // Battery NTC array: 0- RES, 1- MOS (int8_t) (range: -100-150).

unsigned short cell_vol[15];      // The voltage of 15 batteries inside the battery.


};


};  // module dds_

};  // module msg
```

```
};  // module unitree_go


#endif  // __unitree_go__msg__bms_state__idl__
```

The PDF has been downloaded. Rate us: